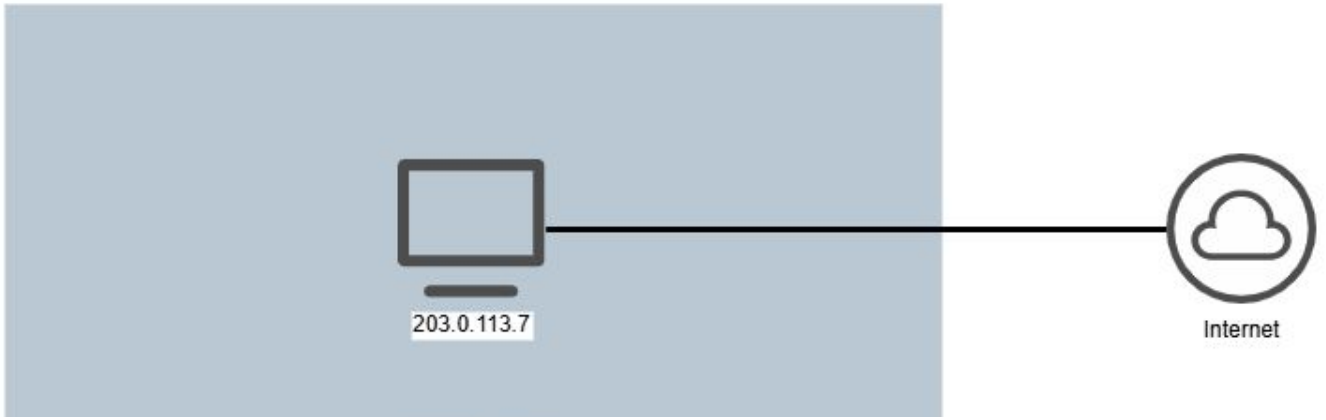
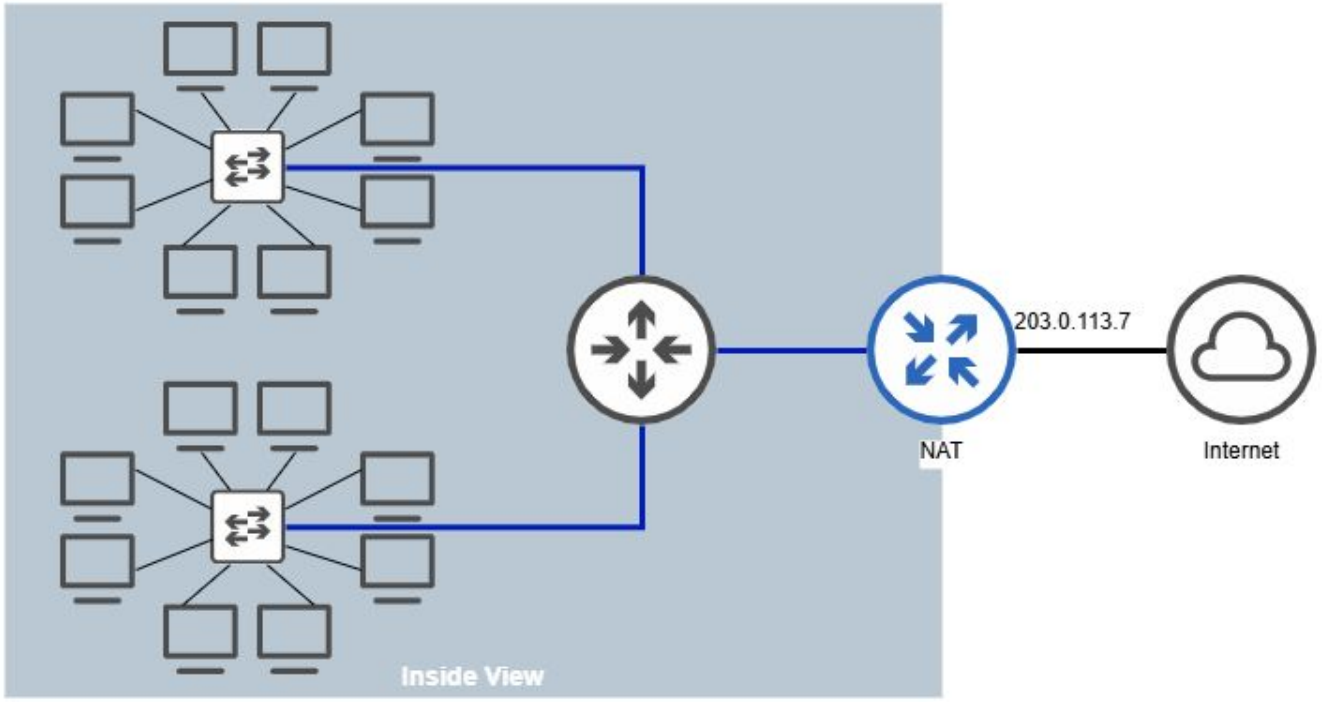
NAT/PAT

NAT maps multiple private IP addresses within a home or organization to a single public IP address and vice versa - effectively hiding a network of devices behind a single IP address.





NAT/PAT

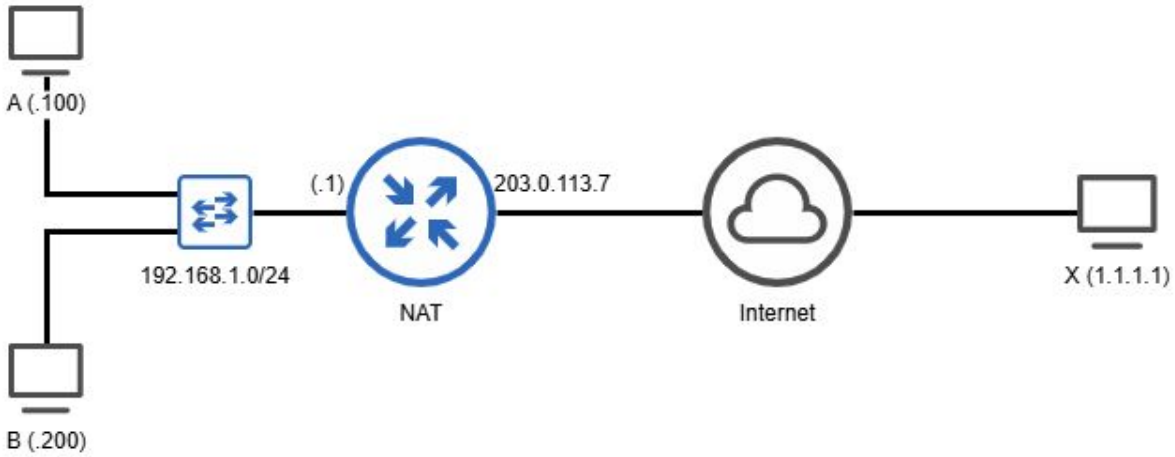
Consider an office with multiple computers - each of which needs a unique IP address. With IPv4 address being 32 bits, we can't assign a globally unique IP address to each computer or device (with 128-bit IPv6 addresses - we can, *but we might not want to!*).

So we assign private IP addresses within the organization. When any of the devices within the org needs to talk to another device on the Internet, it goes via NAT which converts the private IP to a public IP for outgoing packets and vice versa. A single public IP is shared and used for external communication amongst all the devices within the organization.

How does NAT work

Consider two computers with IP addresses 192.168.1.100 and 192.168.1.200 within the organization. The public IP used by the organization, let's say, is 203.0.113.7. The NAT is configured as the default gateway for both A and B. A simplified topology may be as follows -

NAT/PAT



When A (with a private IP) sends a packet towards X (having a public IP), here's what happens –

- A generates packet with Source as 192.168.1.100:1234 and Destination as 1.1.1.1:80
- Packet reaches the NAT box (since it is the default gateway)
- NAT allocates a new source port, say 49152, and creates the following tracking entry in the NAT table

[Internal] 192.168.1.100:1234 <--> [External] 203.0.113.7:49152

- **NAT modifies the source IP and source L4 Port** in the packet. The destination IP and L4 Port is NOT modified. So, the (translated) packet that goes out of the NAT box towards X via the Internet has the following IP and Port Numbers -

Src 203.0.113.7:49152
Dst 1.1.1.1:80

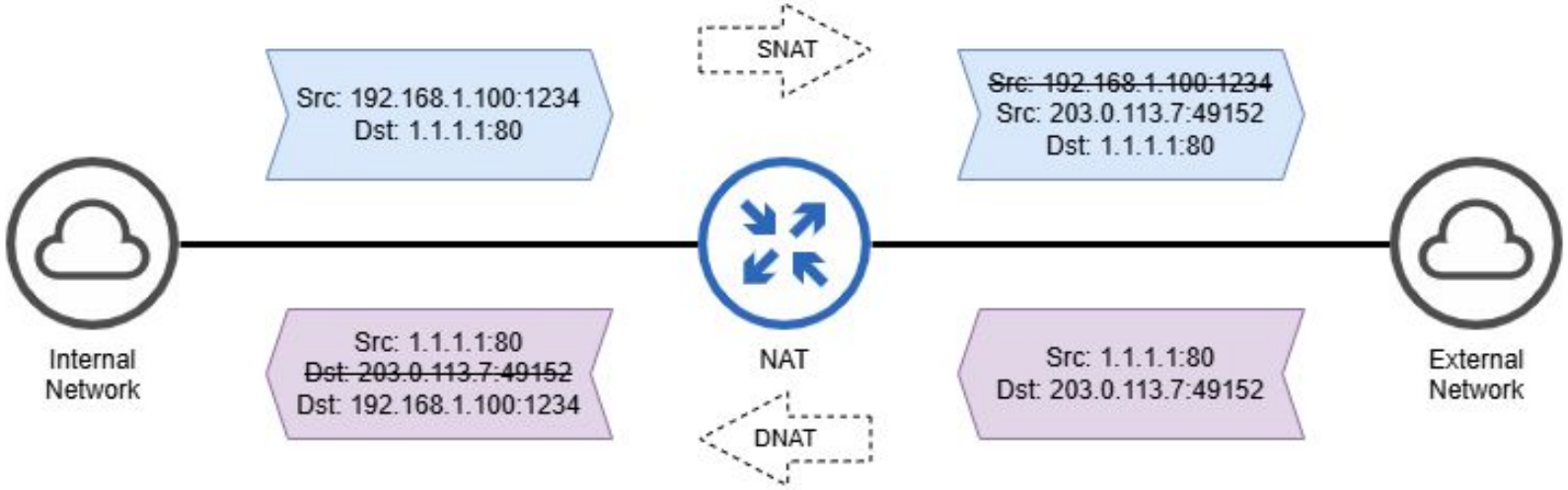
This is called **SNAT or Source Network Address translation and is performed for outgoing flows** (internal to external).

NAT/PAT

Let's see what happens with the reverse flow i.e. incoming flow –

- X sends a reply packet with Source as 1.1.1.1:80 and Destination as 203.0.113.7:49152
- When NAT receives this packet, it looks up the external IP/Port tuple in the NAT table matching the destination i.e. 203.0.113.7:49152
- The lookup results in the internal tuple 192.168.1.100:1234
- NAT modifies the destination IP/port in the packet to this internal tuple
- NAT sends packet on the internal network towards A with Source as 1.1.1.1:80 (unmodified) and Destination as 192.168.1.100:1234

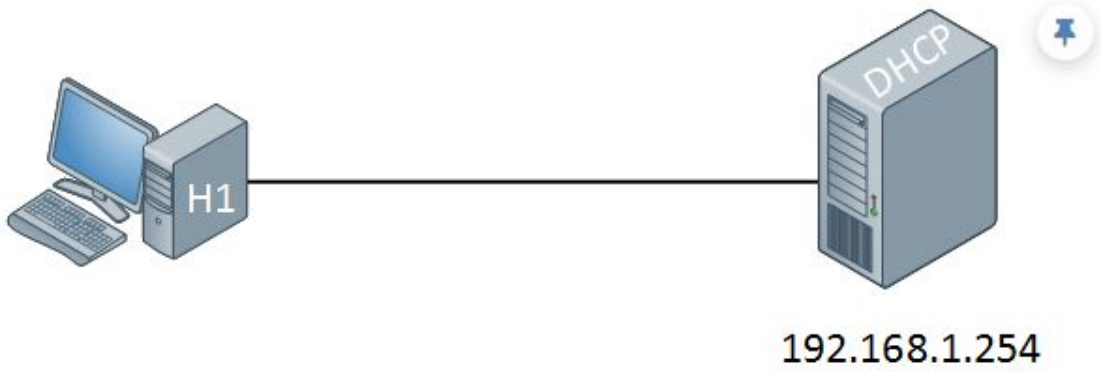
This **mapping of incoming flow is termed as DNAT or Destination Network Address Translation**.



DHCP

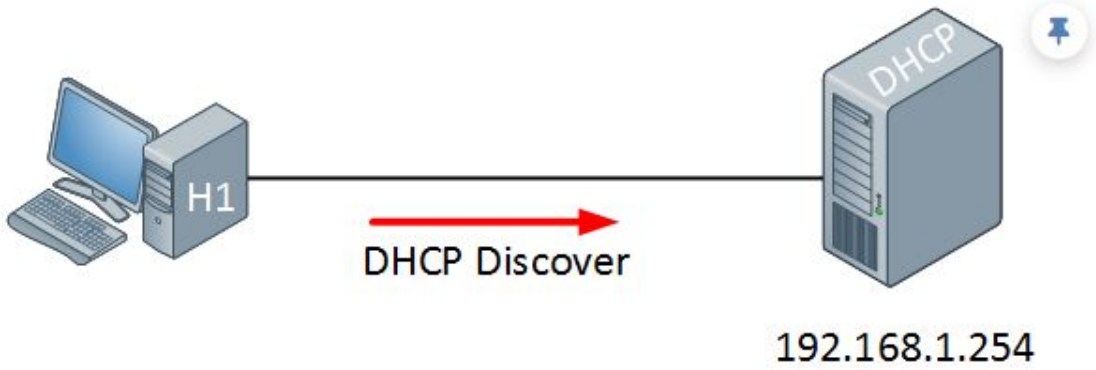
DHCP, or Dynamic Host Configuration Protocol, is a networking protocol that automates the assignment of IP addresses, subnet masks, gateways, and DNS servers to devices in a network. It simplifies the complex task of IP address management, making it easier to connect and maintain devices in a network.

Let’s take a look at the following picture:

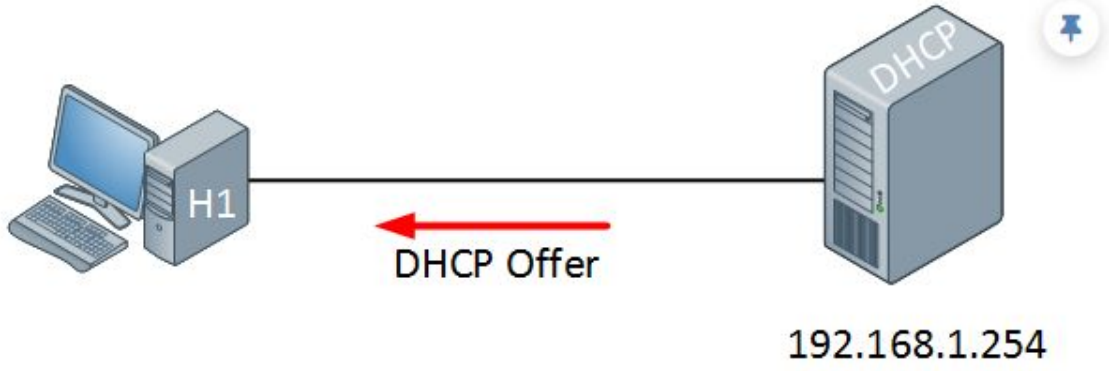


On the left side we have a computer without IP address. On the right side there’s a DHCP server configured with static IP address 192.168.1.254. This DHCP server will supply an IP address to our computer, this is how it works:

DHCP

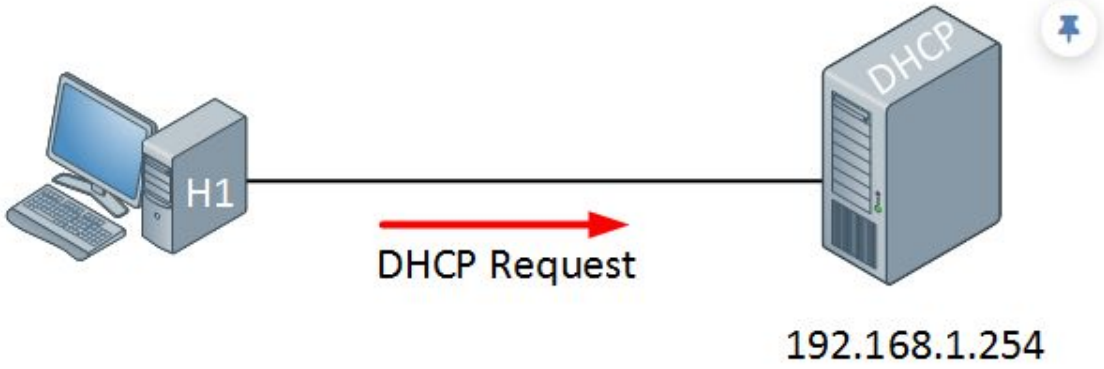


The computer will send a **DHCP discover** message. This is a broadcast because it doesn't have an IP address and it doesn't know if there is a DHCP server on the network. Of course in our scenario we do have a DHCP server so it will respond to this broadcast as following:

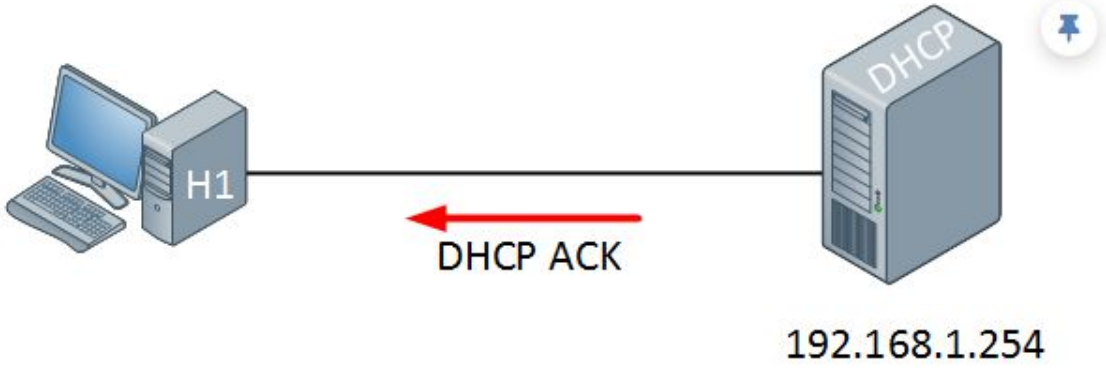


DHCP

The DHCP server will respond with a **DHCP offer** message which contains an IP address for the computer (we have to configure the DHCP server to define which IP addresses we want to give). If we want we can also assign a default gateway and DNS server(s) to the computer. The computer will respond to this information:



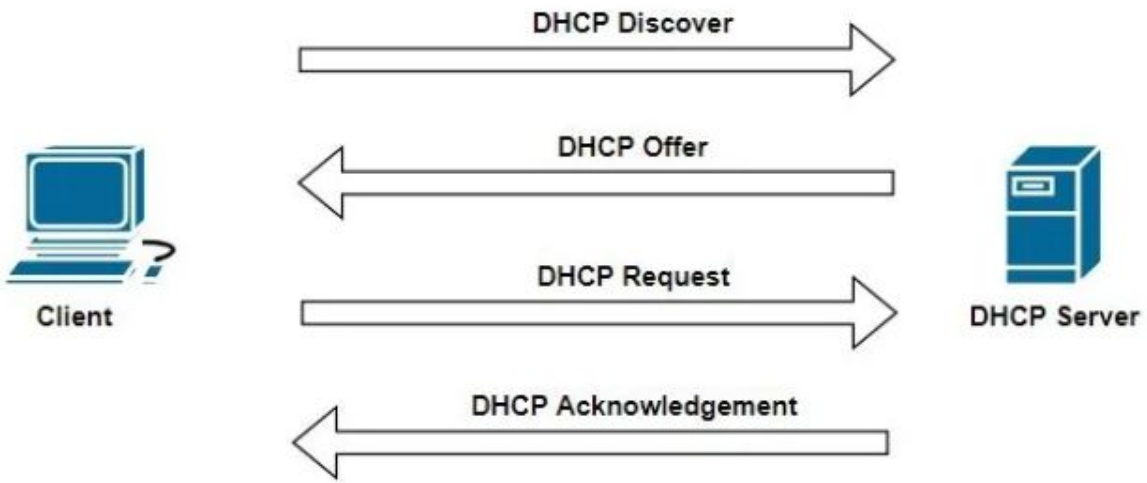
The computer will send a **DHCP Request** in response to the DHCP offer message, asking nicely if it's OK to use the information that it has received. Our DHCP server will respond to this as following:



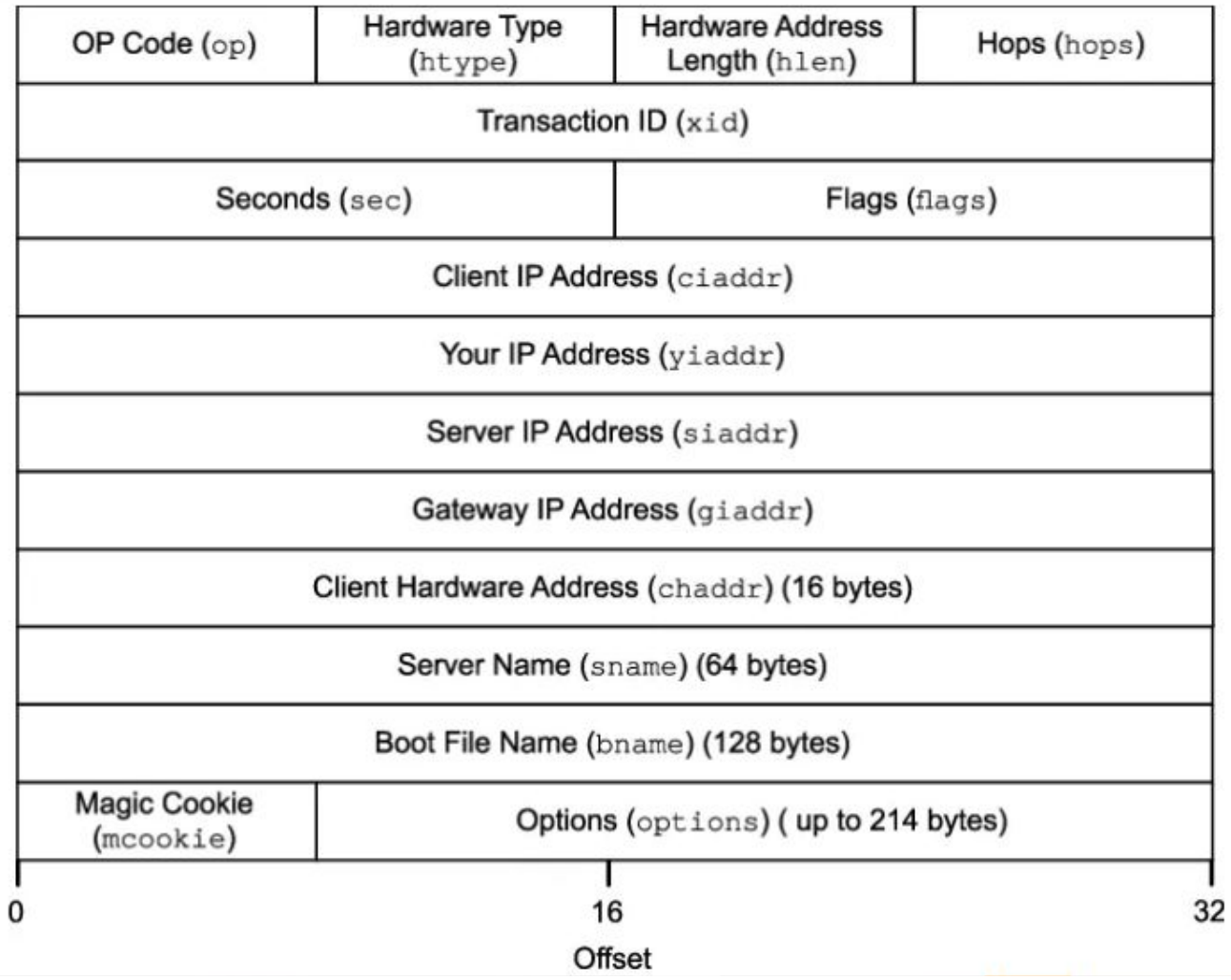
The DHCP server will respond with a **DHCP ACK** message to tell the computer it's OK to use this information.

DHCP

The Four Steps of DHCP Four-Way Handshake



DHCP Packet Format



DHCP

OP Code (8 bits)— this field indicates whether this DHCP packet is a request or a response packet. In DORA, the *Discover* and the *Request* are consider as request packet; meanwhile, the *Offer* and *Acknowledge* are consider as response packet. The value **0x01** represents **request**, while **0x02** represents **response**. For example, if we examine the OP Code of a DHCP Discover, we will find 0x01 in this field; whereas we will find 0x02 if we examine a DHCP Acknowledge packet.

Hardware Type (8 bits) — this field indicates the type of Layer 2 protocol that is using the DHCP.

Hardware Address Length (8 bits) — this field indicates the length of the hardware address. This 'hardware' here refers to the 'hardware type' specified earlier

Hops (8 bits) — this field indicates the number of hops the DHCP DORA packet can travel before being discarded. When a packet traverse from one network to another, it is consider 1 hop count. Hop count is similar to the time-to-live field in IP header; this value is decremented by 1 every time a packet traverse across a router (to prevent packet looping).

Transaction ID (32 bits) — this number is used to identify the DORA's ID. This is an arbitary number generated by DHCP protocol. Each round of DORA will share the same transaction ID.

Seconds (16 bits) — this field indicates the elapsed time since the client ask for an IP.

DHCP

Flags (8 bits) — this field indicates the flags option in the DHCP header. We are only concerned with the first bit among these 8 bits. The first bit indicate if it is a broadcast or a unicast. If the first bit is 0, it is a broadcast; if it is 0, it is a unicast. This means that we will see **1**000 0000 for a broadcast; and **0**000 0000 for a unicast.

Client IP (32 bits) — all IPv4 is 32 bits long. This field indicate the IP address of the DHCP client.

Your IP Address (32 bits) — this field hold the IP address that is being offered by the server to the client.

Server IP (32 bits) — this field indicates the IPv4 address of the DHCP server.

Gateway IP (32 bits) — this field indicate the gateway IP address in LAN to reach a remote DHCP server.

Client Hardware (16 bytes) — this field indicates the MAC address of the DHCP client. In our case, this field is set to **00:01:1C:E8:23:21**.

Options (8–214 bytes) — the options field allows DHCP to specify what types of DHCP messages that is being encapsulated in the DHCP packet.

Routing

- **What Is Routing?**

Routing is a process by which we are selecting the path between one or more networks. Routing selects the path for internet Protocol (IP) packets when they are traveling from source to destination. These routing decisions are made by a spatial type of device which is known as a router, or we can say a network layer (Layer 3 (L3)) device.

- **Why Do We Need Routing?**

We need routing because when we want to communicate between two broadcast domains. That means the source does not know about the destination or we can say that source does not know where the destination is located. To locate the destination, we need routing.

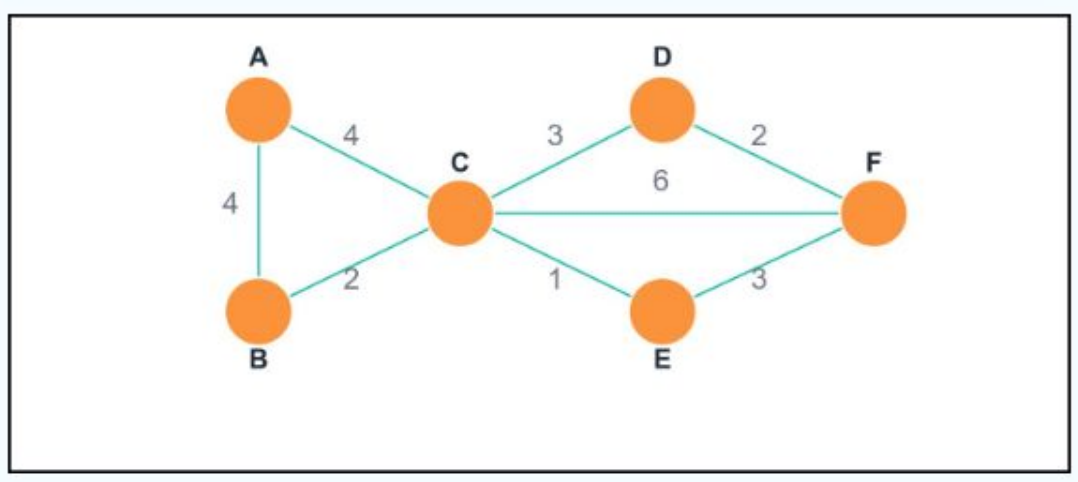
Static Routing	Dynamic routing
1. The network administrator must do the route entry in the routing table. 2. More overhead to a network administrator. 3. Less CPU utilization. 4. It does not use a complex algorithm. 5. Implemented in a small network. 6. Additional resources are not required.	1. Routing entry is done by routing protocol. 2. Less overhead to a network administrator. 3. High CPU utilization 4. It uses a complex algorithm. 5. Implemented in a large network. 6. Additional resources are required.

Dijkstra's algorithm

- Dijkstra's algorithm (or Dijkstra's shortest path algorithm) is used to find the **minimum distance** from a starting node (source) to every other node.

- The algorithm is **used by routers in link-state routing protocols** (like OSPF and IS-IS) to determine the lowest-cost path for data packets. Routers build a map of the network topology, **treating routers as nodes and links as edges** with costs, then calculate the shortest path tree to all destinations.

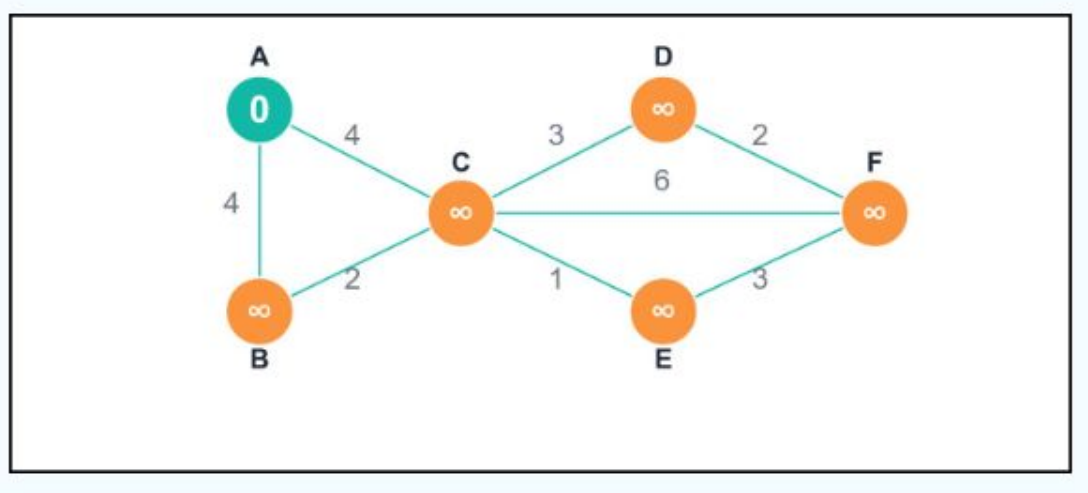
- Let's see how it works step-by-step with this example graph. We'll use Dijkstra algorithm to find the shortest route from A (source node) to every other node in the graph.



Dijkstra’s algorithm

Step 1: Initialization

First, the initial distances from the source node (A) to all other nodes in the graph are established. The distance from the source to itself is set to 0, while the distances to all remaining nodes (B, C, D, E, F) are initialized to infinity (∞), indicating that they are not yet reachable. The algorithm then begins its exploration from the source node.



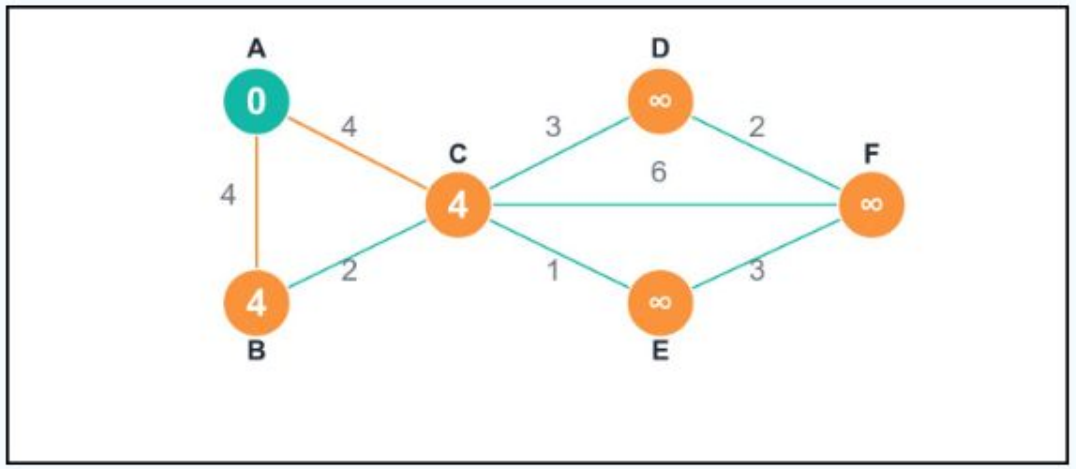
Dijkstra's algorithm

Step 2: Processing Node A

From node A, the algorithm examines all directly connected neighbors. Node A has edges to nodes B and C, each with a weight of 4. The tentative distances to these nodes are calculated as follows:

- Distance to B = 0 + 4 = 4
- Distance to C = 0 + 4 = 4

Since these distances are smaller than their previous values (∞), they are updated. The remaining nodes (D, E, F) retain their initial distances of ∞.



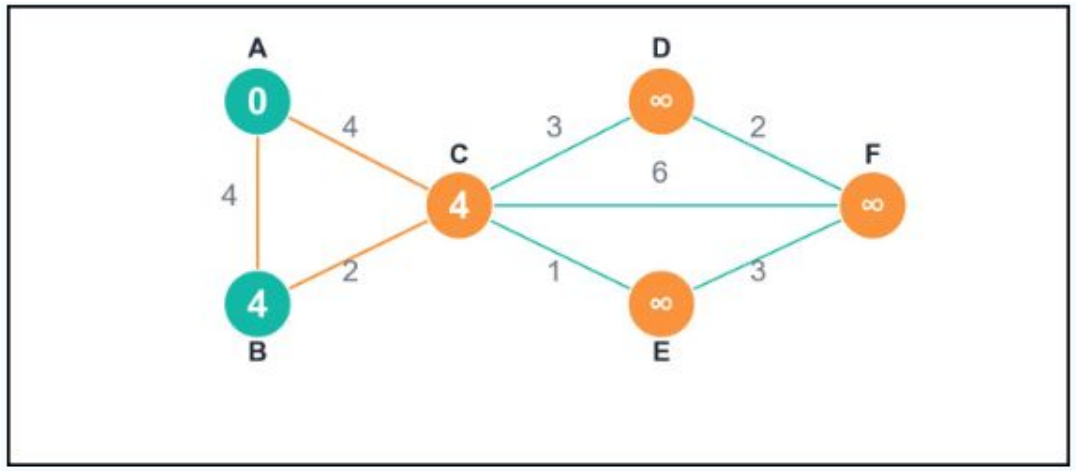
Dijkstra's algorithm

Step 3: Processing Node B

Next, the algorithm selects the unvisited node with the smallest known distance. Both B and C have a distance of 4, so the algorithm may choose either; suppose it selects B.

From node B, the algorithm inspects its unvisited neighbors. B connects to C with an edge of weight 2. The tentative distance to C through B is 4 + 2 = 6.

Since 6 is greater than C's current distance (4), no update is made. The algorithm then proceeds to the next unvisited node with the smallest distance, which is C.



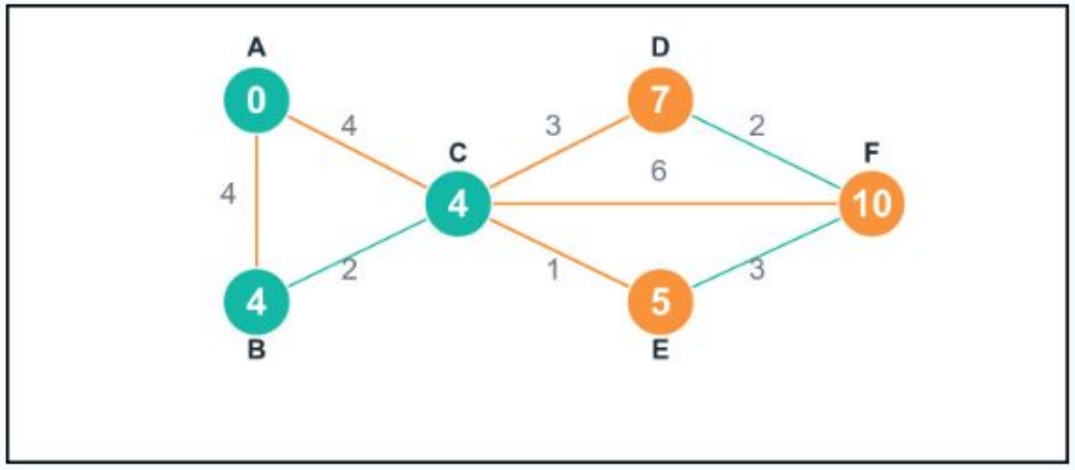
Dijkstra's algorithm

Step 4: Processing Node C

From node C (distance 4), the algorithm examines its unvisited neighbors: D, E, and F. The tentative distances are calculated as:

- Distance to D = 4 + 3 = 7
- Distance to E = 4 + 1 = 5
- Distance to F = 4 + 6 = 10

All of these values are smaller than the current distances (∞), so they are updated accordingly.

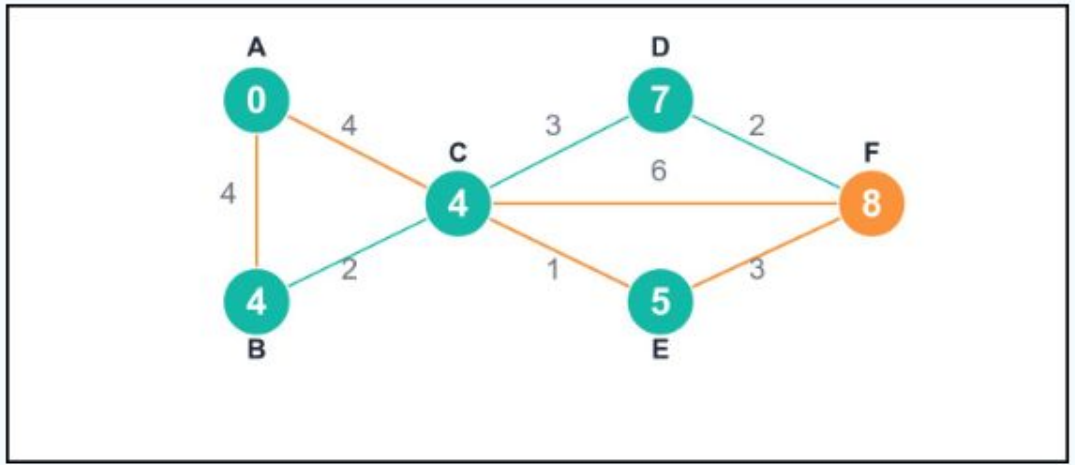


Dijkstra's algorithm

Step 5: Processing Node E

The next unvisited node having the smallest distance is E (distance 5). From E, the algorithm checks its unvisited neighbors D and F:

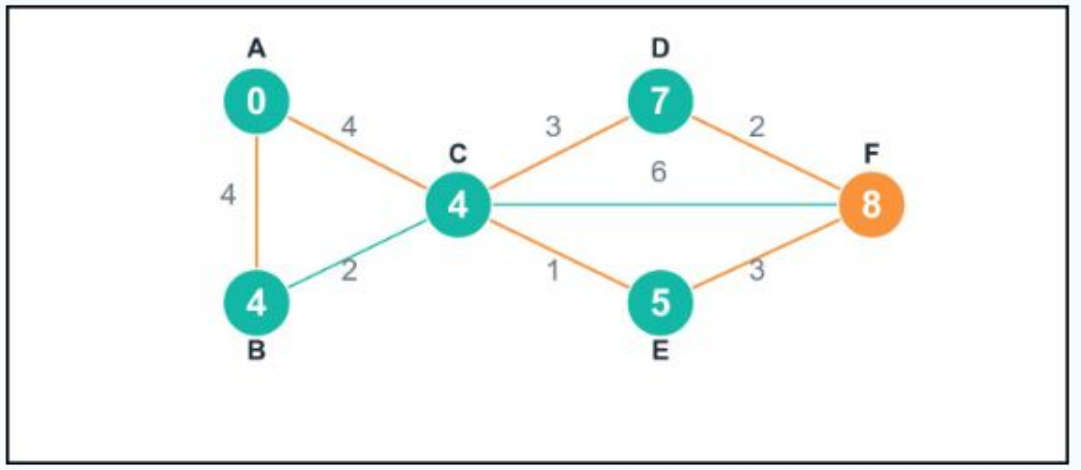
- Distance to D via E = 5 + 3 = 8 → not smaller than the current D (7), so no update.
- Distance to F via E = 5 + 3 = 8 → smaller than the current F (10), so F is updated to 8.



Dijkstra's algorithm

Step 6: Processing Node D

The next node with the smallest distance is D (distance 7). Its only unvisited neighbor is F, with an edge weight of 2. The distance to F via D is 7 + 2 = 9. Since 9 is not smaller than the current F (8), no update is made.

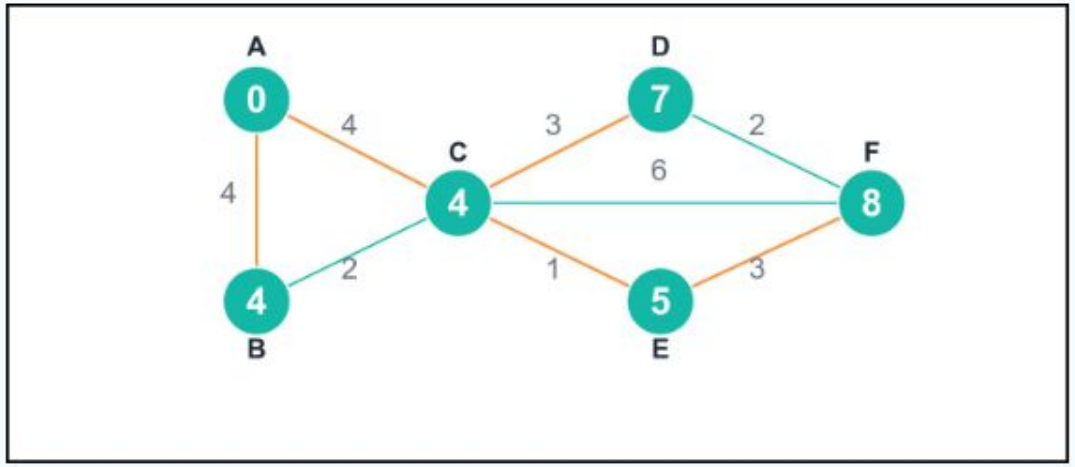


Once all nodes have been visited, the algorithm terminates. The final shortest distances from the source node (A) to all other nodes are:

- A = 0
- B = 4
- C = 4
- D = 7
- E = 5
- F = 8

Dijkstra's algorithm

These distances represent the shortest path tree originating from node A, showing the minimum cost to reach every other node in the graph.



These are the shortest distances from source node A to all destination nodes.